# Bitcoin Commons: A Cryptographic Governance System for Bitcoin Development

BTCDecoded

2026




**Abstract**

Bitcoin's consensus rules are enforced by the network. Development of the software people run is still a maintainer process.

This paper describes two pieces of work. **BLVM** is an independent node checked against a written spec. **Bitcoin Commons** publishes the rules for how that software changes, and those rules can be forked.

The work is organized around six core engineering tracks in the public BTCDecoded organization (spec, consensus, protocol, node, SDK, and governance), plus supporting repositories for packaging, **blvm-spec-lock** verification, and the **blvm-commons** Governance App. Engineering detail lives in technical documentation.

## 1. Introduction

Bitcoin solved Byzantine consensus between strangers (Nakamoto, 2008) but ignored consensus between developers. The network's substantial market capitalization demands institutional maturity matching its technical excellence.

The original cypherpunk developers focused on eliminating trusted third parties in transactions but inadvertently created trusted parties in development. Bitcoin Commons addresses Bitcoin's most critical vulnerability: governance asymmetry between technical consensus and development coordination.

### 1.1 The Talent Bottleneck: Orders of Magnitude and Sources

Bitcoin development draws on multiple hard domains simultaneously (C++, applied cryptography, distributed systems, security engineering, economics/game theory, and open-source governance). Each extra domain narrows the pool. Using conservative, sourced baselines and clearly labeled assumptions, the analysis below estimates the rarity of a contributor who combines these competencies and is available to work on Bitcoin:

**Assumptions and sources:**

- World population baseline: ~8.1B (UN DESA, World Population Prospects, 2022 Rev.)
- Global developers: ~30M-47M (range spanning widely cited industry estimates, incl. SlashData and similar studies)
- C++ share of developers: ~15%-25% (range spanning major annual developer surveys)
- Adult numeracy (problem-solving proficiency): on the order of 10%-20% globally (OECD PIAAC cross-country evidence; global extrapolation is approximate)
- Bitcoin Core maintainers: single-digit individuals; contributors: hundreds (public repo statistics)

**Rarity funnel** (indicative, overlapping, not strictly independent):

- Strong college-level math (calculus/linear algebra): 3%-5% of population, resulting in 240M-400M
- Professional developers: ~30M-47M (subset, separate baseline)
- C++/systems competency: 15%-25% of developers → 4.5M-11.8M
- Applied cryptography + Bitcoin protocol literacy: 1%-2% of C++ devs → 45k-236k
- Distributed systems/P2P networking depth: 30%-50% → 13.5k-118k
- Security engineering mindset (memory safety, adversarial thinking): 20%-30% → 2.7k-35k
- Economics/game-theory literacy: 30%-50%, resulting in 0.8k-17.5k
- Open-source governance (review culture, consensus norms): 10%-30%, resulting in 80-5k

- Communication/reliability under public scrutiny: 30%-50% → 24-2.5k
- Availability/alignment to work on Bitcoin: 10%-30% → ~2-750

**Interpretation:**

- Even with generous ranges, the intersection yields on the order of dozens to a few hundred globally available individuals with the full stack to work reliably on Bitcoin's most sensitive layers.
- Public data corroborates scarcity at the tip: Bitcoin Core has hundreds of credited contributors but only a small, rotating single-digit maintainer set. This human bottleneck contrasts with the cryptographic abundance at the consensus layer.

Of 132 Bitcoin Core authors with at least five pull requests and a quality score of at least 0.3, 41 were still active and 91 were not (Bitcoin Governance Research). Counted another way, 7,971 people have contributed since 2010, 72 sustain a merge record, and 5 hold the keys that merge to the main branch.

**Citations** (illustrative anchors): - UN DESA, World Population Prospects (2022) - SlashData, Global Developer Population Trends - Annual developer surveys (Stack Overflow) for C++ usage - OECD PIAAC, adult skills numeracy distributions - Bitcoin Core repository statistics (GitHub)

## 2. Problem Statement

### Technical Reality

Bitcoin's consensus rules are embedded in hundreds of thousands of lines of reference-client C++ without a canonical mathematical specification the ecosystem standardizes on. In Core 32.1 the rules that decide whether a block or transaction is valid are 10,122 lines, 7% of 151,004 lines of implementation. The same five merge keys gate the other 93%: the code that applies and relays those rules, and the wallet, interface, and network code. The dominant reference implementation carries enormous practical weight over how the network evolves. On November 3, 2026, reachable crawls of the Bitcoin network showed about 89% running Bitcoin Core and about 10% running Knots, down from about 22.5% Knots in the 2025 to mid-2026 window. Knots still shares about 95% of that codebase, so about 99% of those nodes ran one implementation lineage. The separate BLAKE2b proof-of-work chain is not in this count. Safe independent implementations and systematic verification remain exceedingly difficult under reference-client-as-spec; Commons publishes layered specifications to address that gap.

Items with broad agreement have still waited for years. Wallet and node separation, UTXO set commitments, and networking upgrades are in that set. The stall is structural.

### Governance Reality

Bitcoin's development governance relies largely on informal social coordination. At network scale, consequences for violations are rarely systematic, dispute resolution is ad hoc, and authority is difficult to audit. Across the full Bitcoin Core history, the top three merger roles account for about 81% of merges. Authorship concentration is about 0.85 on a scale where 1 means one actor, and review concentration is about 0.92. The network itself stays permissionless. The development process is gated by legitimacy, funding, and who can merge. The system is vulnerable to capture through relationships rather than rules. Network analysis reveals structural misalignments between technical development and social infrastructure, creating coordination gaps that prevent effective governance (Hough, 2025). These patterns reflect the paradox of embeddedness in network structures, where relationships can inhibit coordination and reinforce existing power dynamics (Uzzi, 1997). Funding may not flow to projects with strong grassroots activity, and "rich-get-richer" dynamics reinforce existing patterns rather than enabling competition. Commons addresses these gaps through cryptographic enforcement of multisig thresholds, mandatory review periods, and transparent accountability mechanisms.

Unreviewed self-merges fell from 83.9% in 2016 to 4.0% so far in 2026, and the self-merge rate so far in 2026 is 10.9%. Average review depth on maintainer merges rose from about 1 to about 8. The written process still sets no minimum, so the improvement is a custom.

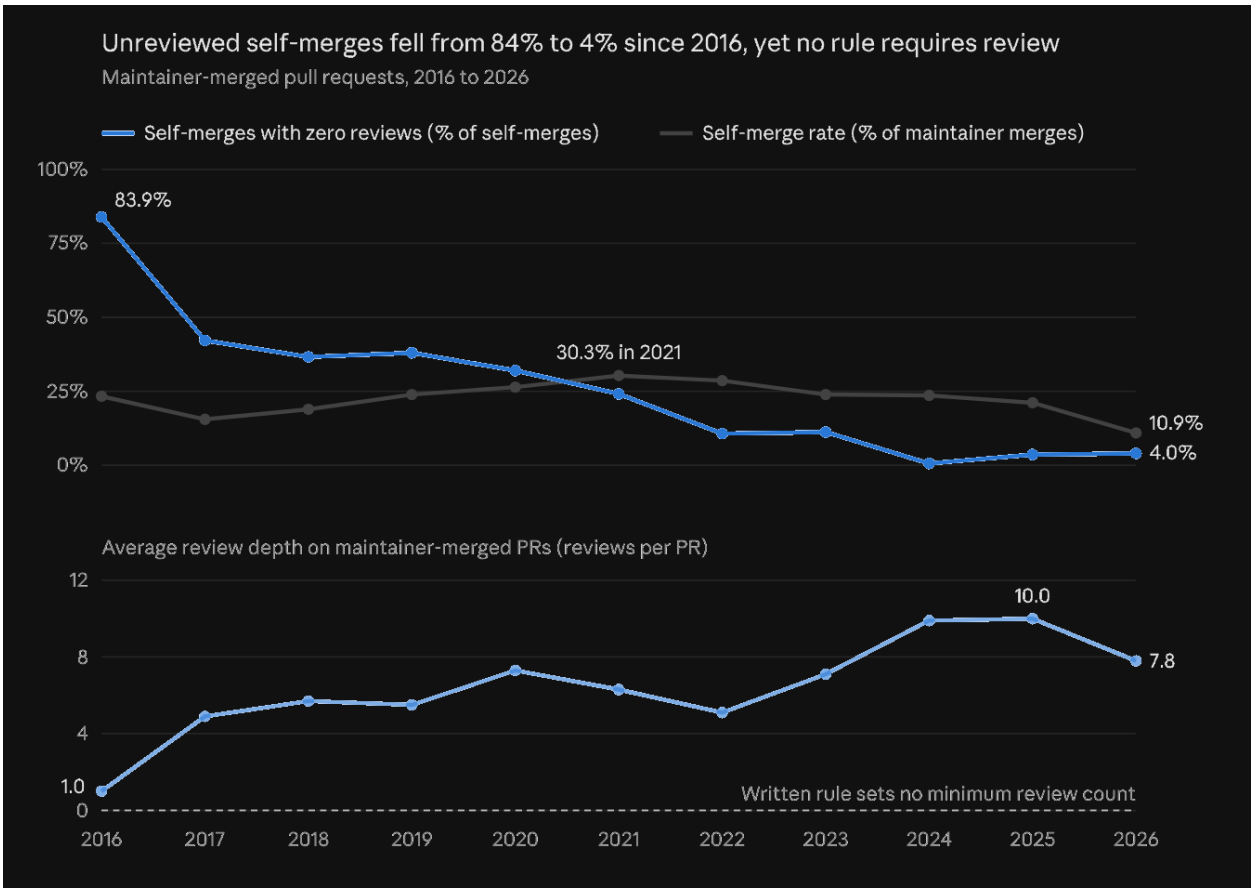



*Figure: Maintainer-merged pull requests, 2016 to 2026. Unreviewed self-merges fell, and review depth rose. The dashed line is the written rule: no minimum review count. Source: Bitcoin Governance Research.*

Scored as a form of governance, the same repository process ties for last. Six measures, each from 1 to 5, put the informal oligarchy that holds the merge keys at 15 of 30, level with a single maintainer. The shortfall against a written oligarchy is legibility and credible commitment. An Ostrom commons and a mixed constitution score higher as forms. Those higher scores are judgments with low confidence. They are not a measurement of Commons in production. Commons is a design aimed at written rules, visible decisions, and exit. Enforcement of that design is not on.

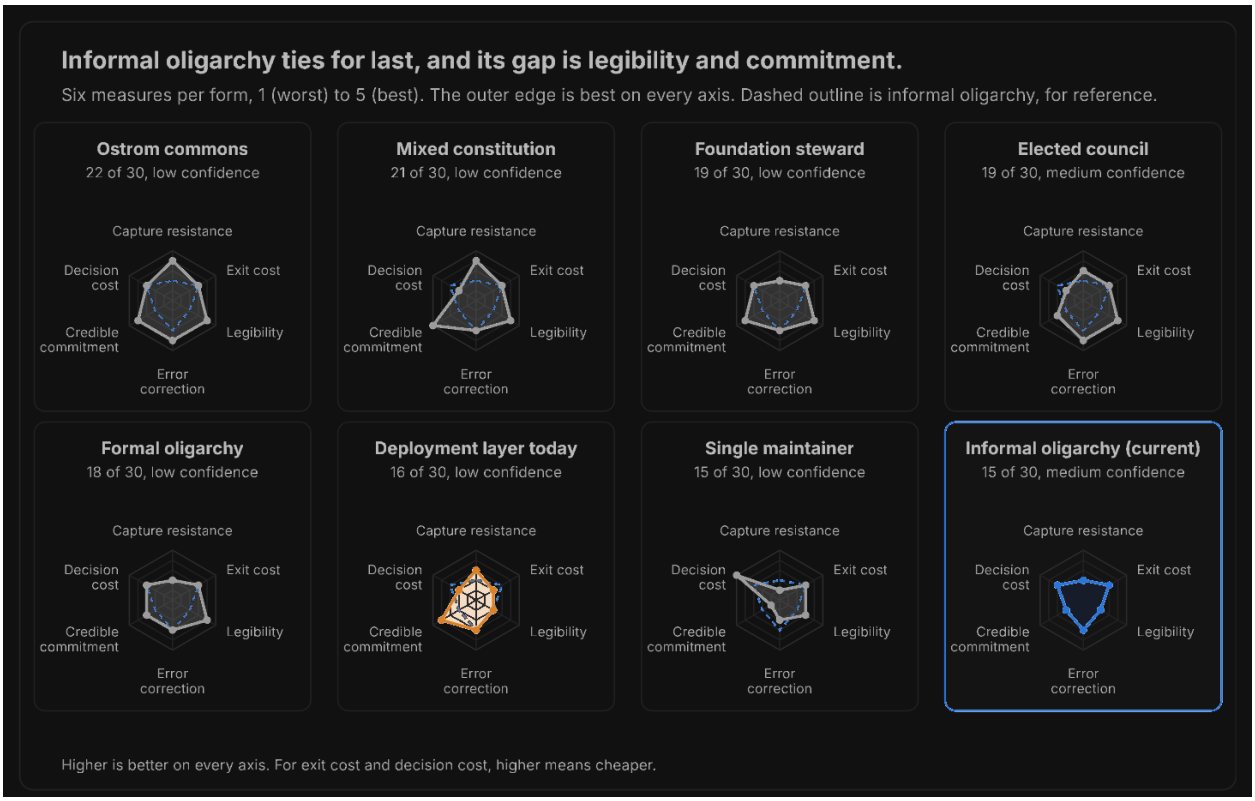



*Figure: Six measures per form, 1 worst to 5 best. The dashed outline is the current informal oligarchy. Higher is better, and for exit cost and decision cost higher means cheaper. Judgments from Bitcoin Has Governance, not a live score for this project.*

### Historical Context

Early leadership transitions and public governance disputes showed that developer coordination can fail without institutional enforcement. Andresen (2014) and Hearn (2016) raised governance concerns from inside development; academic researchers documented the structural gap but offered no implementable remedy (De Filippi & Loveluck, 2016).

### Scale Considerations

Bitcoin's growth from early stages to multi-trillion dollar scale requires institutional reform. The next crisis, whether regulatory capture, internal conflict, or novel attack classes, will not wait for the community to develop governance solutions reactively.

## 3. Theoretical Framework: The Triple Foundation

Bitcoin Commons synthesizes three distinct theoretical frameworks, each addressing weaknesses in the others to create governance architecture stronger than any single approach alone.

### Framework 1: Elinor Ostrom - Commons Governance

Elinor Ostrom won the 2009 Nobel Prize in Economics for proving that shared resources don't inevitably collapse into chaos or capture (Ostrom, 1990). Her research documented principles for governing commons without central authority across centuries of real-world examples.

**Ostrom's (1990) Eight Principles.**

1. **Clearly defined boundaries.** Who may use the shared resource, and who decides.
2. **Congruence between rules and local conditions.** Stricter rules where the stakes are higher.
3. **Collective-choice arrangements.** The people bound by a rule can take part in changing it.
4. **Monitoring.** Participants can see the conditions and the decisions.
5. **Graduated sanctions.** Penalties on someone who breaks a rule scale with the violation.
6. **Conflict-resolution mechanisms.** Disputes have a forum short of leaving the institution.
7. **Minimal recognition of rights to organize.** Outside authorities do not deny the community's right to make its own rules.
8. **Nested enterprises.** Each layer governs itself within the constraints of the layer beneath it.

**What This Provides.** Proven institutional design for shared resources, evidence decentralized governance works, and coordination without hierarchy.

### Framework 2: F.A. Hayek - Spontaneous Order

Friedrich Hayek's Austrian economics provides the competitive discovery mechanism that enables governance evolution rather than rigid design.

**Hayek's Core Insights.** - **Dispersed Knowledge Problem.** No central planner can know what's needed because knowledge is distributed across many actors. - **Competition as Discovery.** Competition reveals information that couldn't be known in advance. - **Spontaneous Order.** Best institutions emerge through evolution, not top-down design. - **Markets Need Infrastructure.** Competition requires actual alternatives to compete.

**What This Provides.** Justification for avoiding central planning, competitive governance discovery, and institutions that evolve through market signals.

### Framework 3: Bitcoin - Cryptographic Enforcement

Bitcoin's innovation provides the enforcement tools that make decentralized governance work at scale without trusted parties.

**Bitcoin's Core Principles.**

- **Don't Trust, Verify.** Cryptographic enforcement replaces social trust.

- **Permissionless Innovation.** Anyone can build without asking permission.
- **Exit Rights.** Fork option provides ultimate check on power.
- **Decentralized Control.** No single point of authority.

**What This Provides.** Tools for enforcing rules without trust, proof decentralized systems work at scale, and a model for implementing Hayek's principles digitally.

### The Triple Synthesis

Each framework covers a hole in the others. Ostrom's commons depended on social pressure, and Bitcoin replaces that pressure with a check anyone can verify. Hayek's competition needs real alternatives, and Ostrom's institutions are how more than one model can exist under rules. Bitcoin settled agreement on the chain and left agreement on the code informal. Hayek's discovery and Ostrom's rules are the design for that layer. The result is a design that is proven, evolving, and enforceable. Enforcement is not on.

### Cryptographic Polycentrism: The Governance Model

Bitcoin Commons implements **Cryptographic Polycentrism**, a governance model that creates multiple centers of power with clear boundaries, enforced through cryptography rather than social pressure. This approach applies Ostrom's proven principles using Bitcoin's cryptographic enforcement tools, enabling competitive discovery through Hayekian market mechanisms while maintaining Ostrom's institutional structure.

Cryptographic Polycentrism differs from traditional polycentrism (multiple decision centers) by using cryptographic enforcement to make boundaries mathematically verifiable rather than socially negotiated. Where traditional governance relies on trust and reputation, Cryptographic Polycentrism uses multisig thresholds, mandatory review periods, and three-layer verification providing real-time transparency and immutable proof to create boundaries designed so that repository administrators cannot bypass them once production governance is on.

This model enables multiple implementations to compete (Hayek's discovery mechanism) while maintaining clear institutional boundaries (Ostrom's principles) through cryptographic proof (Bitcoin's innovation). The result is governance that is both competitive and accountable, proven and evolving, decentralized and enforceable.

Informal governance can work at smaller scale and lower complexity; at multi-trillion dollar scale, social enforcement alone struggles to keep pace with coordination demands and accountability expectations. Commons encodes Ostrom's principles through cryptographic enforcement. The mapping below describes how each principle is implemented in this architecture.

### Mapping The Principles to Implementation

The table is the four-source map from the principles figure: Ostrom, Hayek, Bitcoin, and Cypherpunk. The bullets after it are Ostrom's eight, including congruence and nested enterprises, which that figure did not list as their own rows. Signature checks, Nostr publication, timestamp anchors, and the contributor ladder are designed. They are not on.

The modular architecture implements Ostrom's eight principles through cryptographic enforcement rather than social pressure. The bullets below are the account of each principle in this design.

**Principle 1. Clearly Defined Boundaries.**

- **Layers.** The base consensus layer stays outside the module layer. Modules cannot modify consensus code paths.
- **Contributor jurisdiction.** Standing, when the contributor ladder is in force, applies to code a person has a merged record in. Designed, not enforced.

**Principle 2. Congruence Between Rules and Local Conditions.**

- **Thresholds scale with stakes.** Signature requirements and review periods run from 2-of-3 and 14 days at the extension layer to 6-of-7 and 180 days at the constitutional layers, and 365 days for consensus-rule changes.
- **Status.** Those numbers are configured in the governance repository. Cryptographic enforcement of them is designed, not on.

**Principle 3. Collective-Choice Arrangements.**

- **Binding rules.** Rules that bind every participant change through the governance tiers.
- **Exit, not a tally.** An operator composing a module stack binds only that operator. Hirschman (1970) called this exit. Adoption figures report what operators chose, and no governance decision tallies them.
- **Implementation.** Module composition is declarative (node and SDK configuration). Operator-facing UX may include a CLI, config files, RPC, and an optional UI as tools mature.

**Principle 4. Monitoring.**

- **Cryptographic transparency.** Governance actions are designed to be signed, published to Nostr, and anchored with OpenTimestamps, so any participant can audit them without trusting a monitor. A merge that lacks its required signatures is designed to be refused at the gate. Three-layer verification is the mechanism. Not enforced today.

**Principle 5. Graduated Sanctions.**

- **Penalties on violators.** Sanctions attach to a participant who breaks a contributor obligation, and they scale from formal censure to loss of standing. They are specified with the contributor-standing ladder. Not enforced today.
- **Not the signature schedule.** Multisig thresholds are congruence (Principle 2), not sanctions.

**Principle 6. Conflict-Resolution Mechanisms.**

- **Modules.** Conflicts between modules resolve through operator configuration. No central arbiter. Module conflicts do not threaten consensus.
- **Shared layers.** A contested change can be paused for inquiry. Inquiry is a pause, not a veto, and it is designed, not enforced. A governance fork is the resolution when a dispute cannot be settled inside the rules.

**Principle 7. Minimal Recognition of Rights to Organize.**

- **What the principle asks.** Outside authorities leave the community's rule-making alone. Commons cannot obtain that recognition in every jurisdiction.
- **Substitutes.** Keyholders distributed across jurisdictions, revenue that does not depend on one funder, and a ruleset any participant can fork.

**Principle 8. Nested Enterprises.**

- **Layers inside one ruleset.** Modules operate inside the consensus boundary. The extension, application, implementation, and constitutional layers each carry their own thresholds inside one ruleset.

Commons implements all eight Ostrom principles through Cryptographic Polycentrism, encoding boundaries, congruence, nesting, and auditability in architecture rather than social pressure alone.

## 4. Technical Solution: The Orange Paper

### Problem

Bitcoin's consensus rules lacked a canonical mathematical specification industry-wide (reference-client-as-spec remains the practical default). The consensus-critical denial-of-service vulnerability CVE-2018-17144 existed undetected for years: exactly the class of error formal verification and layered testing aim to prevent. Forking the reference client keeps its codebase and its merge process. Independent clients that dropped the UTXO set, or that stopped active development, do not give operators a second consensus implementation.

### Solution

The Orange Paper provides a formal mathematical specification of Bitcoin's consensus protocol through AI-assisted extraction from Bitcoin Core's codebase. The specification includes:

- Mathematical foundations (set theory, cryptographic primitives, network protocols)
- State transition functions (block validation, transaction validation, consensus rules)
- Economic model (mining rewards, fee calculations, difficulty adjustment)
- Security properties (Byzantine fault tolerance, Sybil resistance, double-spending prevention)

### Benefits

- **Safe alternative implementations**: Independent implementations can verify against mathematical specification
- **Formal verification**: Consensus-critical code is checked against the Orange Paper through **BLVM Specification Lock**, property tests, and CI; see Formal Verification
- **Fewer unnamed rules**: a written spec makes a consensus mistake easier to point at

### AI-Assisted Extraction Methodology

The Orange Paper uses AI-assisted extraction from Bitcoin Core's codebase to formalize consensus rules. This approach: - Analyzes Bitcoin Core's codebase to identify consensus-critical code paths - Extracts mathematical relationships from implementation details - Creates formal specifications that are independent of specific code structure - Enables verification that specification matches actual network behavior

### Proof Maintenance and Specification Quality

The formal verification process includes ongoing maintenance so the specification stays aligned with the implementation. A code change is checked for drift against the Orange Paper, the spec is updated when the wording and the code diverge, and the lock is run again.

Commons publishes a **layered specification**: the **Orange Paper** (formal math), **CONSENSUS_SPEC** (numbered RFC 2119 rules mapped to code), and **PROTOCOL** (extended constants and formulas). Entry points are at thebitcoincommons.org/spec.html. **blvm-spec-lock** ties `blvm-consensus` to these artifacts; drift and coverage tooling tracks alignment between spec wording and lockable contracts. Details: Orange Paper, Formal Verification.

Parallel independent specification efforts reinforce the same goal from complementary formalisms. Toby Sharp's Hornet project demonstrates that a minimal, executable consensus specification and domain-specific language can sync mainnet independently of the reference client (Sharp, 2025; hornetnode.org), strengthening the case for Commons' layered Orange Paper and CONSENSUS_SPEC approach. Keagan McClelland's **btc-verified** pursues the same correctness goal through machine-checked Lean 4 proof leaves over serialization, transaction identifiers, and merkle commitments,

validated against mainnet golden vectors (McClelland, 2026; github.com/ProofOfKeags/btc-verified). Where Hornet emphasizes an executable implementation-neutral specification, btc-verified emphasizes theorem-checked byte-layer foundations; Commons' spec-lock contracts and differential testing complement both. The choice of notation is a governance choice. A reader of the Orange Paper needs the mathematics and the domain. A reader of a Lean proof needs Lean. A reader of a custom language needs that language.

#### Empirical consensus alignment (differential testing and fuzzing)

Mathematical specification and spec-lock contracts bound *what the rules ought to be*. **Differential testing** and **fuzzing** ask whether the running implementation agrees with the dominant reference implementation on real chain history and on adversarial inputs. Methodology, current depth, and CI status are documented in technical documentation. Prefer that over this document for live coverage figures.

Together these layers complement formal methods: **specs and contracts** for specification fidelity; **replay and fuzzing** for empirical alignment with observed network behavior.

#### BLVM Architecture

BLVM (**Bitcoin Low-Level Virtual Machine**) is an implementation checked against the Orange Paper. The paper is the written form of the consensus rules, so more than one implementation can be compared to it. Stack layout: System Overview, Consensus Overview.

**Single Source of Truth.** All consensus logic resides in `blvm-consensus`. Upper tiers (`blvm-protocol`, `blvm-node`) delegate validation calls with no duplicate implementations. Path dependencies ensure changes propagate immediately through Rust's type system.

Because every consensus decision flows through `blvm-consensus`, verification and tests at that layer bound behavior for the whole stack: upper tiers delegate validation and cannot bypass that layer.

## 5. Architectural Solution: Modular Governance

**BLVM** is the written spec and the node. **Commons** is the published governance. The modular architecture is where they meet. Commons distributes authority through Cryptographic Polycentrism, defined below.

Commons distributes authority through cryptographic multisig requirements and mandatory review periods, designed so that no single keyholder can approve a merge. This Cryptographic Polycentrism model creates multiple decision centers (repository layers, action tiers, emergency tiers) with mathematically enforced boundaries. Approval and blocking are different costs, as Section 6 sets out.

### Two-Layer Architecture

The modular architecture consists of two layers that transform governance conflicts from political battles into architectural choices:

#### Layer 1: Mandatory Consensus (Base Node)

- Bitcoin's consensus rules, unchangeable without network agreement
- Cryptographically enforced, defines what "Bitcoin" means
- Examples: block validation, transaction validation, fork choice rules

#### Layer 2: Optional Modules (Extension System)

- User-controlled features that can be enabled or disabled
- Communities can fork/modify/compete, user choice determines winners
- Examples: Lightning Network, Taproot Assets, privacy enhancements, Stratum/mining-pool interfaces, optional merge mining where enabled

### Revenue Model

- Self-sustaining development through optional modules
- Three complementary channels; no single stream is required: **optional modules** (operational today), **module registry and distribution** (not live yet; discovery and fees when deployed), **governance announcement zaps** (Lightning), and **optional merge mining** for pools that choose secondary-chain revenue
- Reduces reliance on grants, donations, or external funding
- Aligns incentives with usage and adoption, reducing capture risk

### Module Isolation

Optional features load as **modules**: separate processes with strict boundaries, not compile-time features baked into the base node:

**Process Isolation Mechanisms:** - Each module runs in its own process with isolated memory - Modules communicate through a documented **ModuleAPI** (subprocess IPC) - Base node validates all blocks using Orange Paper specification regardless of enabled modules - Module state is separate from consensus state (UTXO set)

**API Boundaries:** - Modules interact with the base layer only through documented interfaces - No direct access to consensus functions or core data structures - Module failures cannot propagate to the base node

**What modules CANNOT do**: Modify consensus rules, alter block validation, cause network splits

**What modules CAN do**: Provide optional services (relay, mining interfaces, Lightning, ZMQ, etc.) without affecting base-node stability

**Containment Strategy**

The modular architecture satisfies both camps simultaneously:

- **"Don't Change Bitcoin" Camp**: Gets pure Bitcoin base layer with no modifications
- **"Make Bitcoin Useful" Camp**: Gets optional features through modules
- **Miners and pools**: Stratum V2 connectivity to the ecosystem; optional merge mining only where operators enable it, not the primary sustainability bet

**The Module System IS The Governance System:** Instead of governing through committees deciding features, Commons governs through architecture enabling choice. The module system isn't just technical: it's the governance mechanism itself, implementing exit through user configuration (Hirschman, 1970), Hayek's competitive discovery through module competition, and Bitcoin's permissionless innovation through fork-ability. Collective choice over rules that bind every participant remains with the governance tiers in Section 6.

**Fragmentation Analysis:**

Governance forks preserve the consensus layer while allowing governance changes. Users can fork governance rules while keeping the same Bitcoin consensus. This is the ultimate accountability mechanism. **Alternate full-node clients** (for example Bitcoin Knots) can share one chain when validation rules stay aligned. Knots is mostly the same codebase, so that coexistence is not a second implementation. Reported shares move by window and method. The structural point is one chain, not a market-share figure.

**Forking Process:** Governance rulesets can be exported as versioned, signed packages containing action tiers, maintainers, cryptographic proofs, and semantic versioning. This enables users to fork governance while maintaining compatibility and verification.

Optional **modules** are operational today (loadable processes via ModuleAPI). A **module registry** for permissionless listing and automated validation is part of the distribution model but **not live yet**.

## 6. Cryptographic Governance Enforcement

*Scope:* Merge gates, production keyholders, and full three-layer transparency are **designed**; **production activation** follows prerequisites documented in the governance repository. This section describes intent, not guaranteed live operation.

Commons implements Cryptographic Polycentrism through three complementary verification layers (distinct from repository layers). The Governance App is the merge gate. Nostr carries signed

status. OpenTimestamps anchors the record. Each layer checks the others. None of the three is enforced in production today.

**Layer 1: Governance App (Merge Control).** *Designed to*, when production governance is activated, enforce merge policy. The Governance App is designed to be the merge gate. It would check signatures and review rules and refuse a merge that does not meet them. Cryptographic merge enforcement is off today. A GitHub App is one way to run that gate. The requirement is the gate, not the forge.

- The Governance App is designed to validate multisig requirements (varies by layer: 2-of-3 to 6-of-7)
- A self-hosted runner behind WireGuard VPN is designed to validate signatures using secp256k1. On GitHub, that runner is a GitHub Actions runner. Another forge would use its own hook.
- Once production governance is on, whoever administers the repository cannot bypass the cryptographic requirements
- Signature validation is designed to happen before merge approval
- Designed to require explicit cryptographic approval from multiple maintainers before any merge
- **No-self-merge:** a contributor cannot cryptographically approve their own pull request; the Governance App is designed to reject self-signed merges. Not enforced today.

**Layer 2: Real-Time Transparency (Nostr).** *Designed to* publish signed status updates (binary hash, config hash, recent merges, health metrics) to Nostr relays so subscribers can verify server integrity. Cadence and alerting are deployment parameters, not guarantees about any particular live deployment.

**Layer 3: Immutable Proof (OpenTimestamps).** *Designed to* anchor canonical registry snapshots and critical events (key rotations, deployments) to the Bitcoin blockchain for dispute resolution and independent verification. Provides court-admissible evidence where applicable.

**Cross-Layer Verification:** Three independent layers verify governance actions and each other. Risk at one layer does not compromise the others. This defense-in-depth approach ensures governance integrity even if one verification method is compromised.

### Emergency Response

Commons implements a three-tier emergency system for critical security and network issues. **Emergency tier activation** is separate from **action tier** classification (see below): emergency keyholders declare the class; merge rules and review periods adjust until automatic expiration.

**Activation (all classes):** 5-of-7 emergency keyholders. **Merge policy while active** varies by class: network-threatening incidents use the fastest path (zero review, lower maintainer merge threshold);

less urgent classes use longer review and stricter merge thresholds. Authoritative parameters: governance repository (`emergency-tiers.yml`).

**Critical Emergency** (Network-threatening vulnerabilities): - Merge signatures while active: 4-of-7 - Review period: 0 days (immediate response) - Maximum duration: 7 days (no extensions) - Examples: Consensus-critical denial-of-service bugs, consensus fork risks, P2P network DoS vulnerabilities, remote code execution

**Urgent Security Issue** (Serious security issues): - Merge signatures while active: 5-of-7 - Review period: 7 days - Maximum duration: 30 days (1 extension allowed) - Examples: Memory corruption vulnerabilities, privacy leaks, crash exploits, privilege escalation

**Elevated Priority** (Important but not critical): - Merge signatures while active: 6-of-7 - Review period: 30 days - Maximum duration: 90 days (2 extensions allowed) - Examples: Competitive response needs, important bug fixes, performance degradation, ecosystem compatibility issues

The system scales **response speed and review strictness** to severity, not a single monotonic rule that stricter always means worse. Network-threatening issues get the fastest merge path; important-but-non-critical work uses longer review and higher merge bars. All emergency actions require post-mortem documentation and automatic expiration.

An emergency expires on its own, and the lowered merge rule expires with it. A change already merged under that rule stays merged. The repository does not bind an activation to one patch, does not require later ratification at the ordinary threshold, and does not bar an emergency from covering governance rules or key sets. Five emergency keyholders can therefore turn a constitutional change into a 4-of-7 merge with no review, and the result remains after the emergency ends. Closing that gap is an open design decision.

#### Security Architecture: Push-Only Design

The layout below is the GitHub deployment of the Governance App. Another forge would keep the same rule, outbound-only checks before a merge, and would change the host.

**Security Architecture Details:**

- **No HTTP Endpoints**: Governance servers have no incoming HTTP endpoints (minimal exposure surface)
- **VPN Isolation**: Servers communicate outbound only through WireGuard VPN
- **Self-Hosted Runner**: On this host, a GitHub Actions runner behind WireGuard VPN validates signatures
- **Data Flow**: Server to the repository host (push) to Nostr (publish) to Bitcoin (anchor)
- **Public Read Access**: Repository, Nostr relays, Bitcoin blockchain (read-only for public)

### Multisig Threshold Details

**Authoritative configuration.** Signature thresholds, repository **layer** definitions, **action tier** names, review periods, and **emergency** parameters described in this section are **illustrative** of the governance *design*. **Security-class overrides** (for example consensus-critical or cryptographic paths) may impose stricter requirements than the tables below; see `action-tiers.yml` in the governance repository. **Authoritative numeric requirements and rule text** live in that repository (YAML and related artifacts) and may change as policy evolves. This PDF is not the source of truth for merge policy; when in doubt, consult the repo.

Commons implements a **5-tier constitutional governance system** that combines two dimensions: **repository layers** (where the change is made) and **action tiers** (what type of change is being made). When both apply, the repository's combination rule selects the longer review period and the larger signature count. That count is not always the stricter condition when the layer and the tier name different key sets.

**Repository Layers (Where):** - **Layer 1-2 (Constitutional)**: Orange Paper and Consensus Proof - 6-of-7 signatures, 180-day review (365 days for consensus changes) - **Layer 3 (Implementation)**: Protocol Engine - 4-of-5 signatures, 90-day review - **Layer 4 (Application)**: BLVM Node - 3-of-5 signatures, 60-day review - **Layer 5 (Extension)**: SDK and modules - 2-of-3 signatures, 14-day review

**Action Tiers (What):** - **Tier 1 (Routine Maintenance)**: Bug fixes, documentation, optimizations - 3-of-5 signatures, 7-day review - **Tier 2 (Feature Changes)**: New RPC methods, P2P changes, wallet features - 4-of-5 signatures, 30-day review - **Tier 3 (Consensus-Adjacent)**: Changes affecting consensus validation code - 5-of-5 signatures, 90-day review - **Tier 4 (Emergency Actions)**: Critical security patches, network-threatening bugs - 4-of-5 signatures, 0-day review (with post-mortem required) - **Tier 5 (Governance Changes)**: Changes to governance rules themselves - 5-of-5 signatures, 180-day review

**Emergency PRs vs emergency activation.** *Action Tier 4* classifies pull requests for critical security work under normal maintainer multisig. *Emergency tier activation* (above) is a separate declaration by emergency keyholders that switches merge rules until expiration. Both can apply together; see `emergency-tiers.yml` and `action-tiers.yml` in the governance repository.

**Combination rule:** When both a layer requirement and a tier requirement apply, the governance repository takes the longer review period and the larger signature count. For example: - A routine bug fix (Tier 1) in the Protocol Engine (Layer 3) takes 4-of-5 signatures and a 90-day review - A new feature (Tier 2) in the SDK (Layer 5) takes 4-of-5 signatures and a 30-day review - A consensus-rule change (Tier 3) in the Orange Paper (Layer 1) takes 6-of-7 signatures and a 365-day review

Six signatures out of seven is a larger count than five out of five and a weaker condition when the five must be unanimous and the sets differ. The repository's combination rule does not yet require both named sets to sign. Whether a change that meets both a layer rule and a tier rule must satisfy

each set, rather than a single merged count, is an open design decision. The lists above are the numbers. The YAML is authoritative.

All signatures verified using secp256k1 (same curve as Bitcoin). The Governance App is designed, when production governance is activated, to validate signatures before allowing merges, so that repository administrators cannot bypass the cryptographic requirements. A GitHub App is one host for that gate.

A high threshold makes approval expensive and blocking cheap. Where a missing signature counts as a no, two constitutional keys can stall a 6-of-7 change, and one key can stall a 5-of-5 governance change or a 7-of-7 security-critical change. A stalled change is a capture outcome in its own right.

The specified remedy separates the two costs. A block is a signed reject of one change, it names a reason, and it expires at the end of that change's review unless the same keyholder renews it. Silence is not a reject. On a consensus-rule change, one explicit reject forces the full review, because a bad merge can split the network and a delay leaves the current rules in place. Two renewed rejects can keep holding that change. On a change to the governance rules or to who holds a key, the bar is a supermajority short of unanimity, and a keyholder whose own seat is the subject is recused. These reject, expiry, and recusal rules are specified. They are not in the governance repository, and they are not enforced.

Shared-layer decisions are binary, one specific change against the status quo. Arrow (1951), Gibbard (1973), and Satterthwaite (1975) showed that no ranked voting rule over three or more alternatives can be non-dictatorial, Pareto efficient, and independent of irrelevant alternatives, and that every such rule rewards some voter for misreporting. A binary decision falls outside those results. They apply again when two rival changes and the status quo are in play together, which is why a reject attaches to one change and does not become a standing veto over the layer.

Mandatory review periods prevent rushed merges. The dual-dimensional system (layers and tiers) keeps routine changes in critical repositories at the higher approval threshold.

**Contributor standing (proposed)**

Beyond repository-layer multisig, a **contributor-standing ladder** complements merge policy: ledger-derived standing, crate-scoped participation, **no-self-merge** enforcement at the merge gate, and an Inquiry mechanism as a pause, not a veto, for contested changes. Described in a separate specification to be published when cryptographic governance activates. **Not enforced today.** The live governance repository defines maintainer multisig and tiers only. Its maintainer lists are placeholders, one set per layer plus a separate emergency set, and the production roster has not been set. The thresholds above settle how keyholders decide. Who holds the keys settles who decides. Tier 5 at 5-of-5 means any incumbent can veto a change to the key set, including their own removal. Selection, rotation, and recusal are outlined only, and they are not enforced.

## 7. Economic Sustainability

### The Funding Gap

Only $8.4 million from 13 organizations supported Bitcoin Core development in 2023, while the network reached a $2 trillion market cap (Hough, 2025). That is roughly **0.00042%** of market cap, or about $4.2 million of protocol funding per $1 trillion of native market cap. On the same normalization, Ethereum protocol funding in the cited window is about $125 million per $1 trillion.

*The funding and market-cap figures are **snapshots** from the cited work (2023 funding year; market cap at the time of that analysis). They illustrate scale mismatch; they are not live dashboards.*

In raw dollars, Bitcoin Core's grant year is not the smallest budget in open source. It is larger than the organizations that keep several widely used projects running, and smaller than a protocol treasury's development line or the Ethereum figure above.

The Apache Software Foundation reported about $2.4 million of revenue in FY2024. The Python Software Foundation reported about $4.1 million of revenue in 2024. The Rust Foundation reported about $4.3 million of revenue in 2024, and about $413,000 of that went out as grants to maintainers. The Zcash Foundation's 2024 expenses were about $8.0 million: about $1.0 million of protocol work and about $5.1 million of community grants. Its block-reward development fund ended at the November 2024 halving. Polkadot's 2024 treasury report counts about $32 million of development spending, of which about $6.7 million was the protocol and SDK. The same report counts about $133 million of treasury spending in total, mostly outreach and business development, so that larger number is not a protocol budget. The Linux Foundation reported about $221 million of revenue in 2024 for the whole foundation. That figure covers many projects, so it is not a kernel budget and it is not on the chart.

### Self-Sustaining Revenue Models

Bitcoin Commons is built to fund operations through optional modules, which are operational today, reducing reliance on grants and donations. Sustainability is **multi-channel**: module revenue, registry-based distribution (when the registry is live), Lightning zaps, and optional mining-side revenue; none is assumed to carry the project alone.

**Module registry and distribution:** Planned permissionless discovery, quality signals, and developer alignment with project revenue; **not live yet**.

**Governance announcement zaps:** Lightning zaps attached to governance announcements fund governance operations directly and tie community support to transparent announcements.

**Optional merge mining:** For pools that enable it, coordinates merge mining for secondary chains as **supplemental** revenue, not a prerequisite for the economic model.

These self-sustaining models align incentives with usage and adoption, making the system less vulnerable to capture than grant-dependent funding models. Commons positions as **infrastructure**

**for multiple implementations**, not a reference-client replacement: success is measured by ecosystem health and adoption of the model, not market share.

## 8. Failure Modes & Mitigations

### Governance Capture

**Risk**: Keyholder collusion or compromise **Mitigation**: Multi-jurisdictional keyholders, transparent operation, fork-ready design. Informal governance at scale is easier to capture through relationships than architecture with enforced boundaries. Capture can also take the form of paralysis. A high threshold makes blocking cheaper than approving. At 6-of-7, two keys can stall a constitutional change. At 5-of-5, one key can stall a governance change.

### Regulatory Pressure

**Risk**: Authorities pressure keyholders to implement backdoors **Mitigation**: Keyholders are distributed so that no single jurisdiction holds enough keys to approve a change at any layer. Blocking is a lower bar. A jurisdiction that can compel two constitutional keyholders can stall a 6-of-7 change, and one that can compel a single keyholder can stall a 5-of-5 governance change. A missing signature is not yet distinguishable from a refusal, so the stall is not fully attributable until rejects are signed. A forkable ruleset is the remedy for a stall that persists. Modular containment limits the blast radius of a compelled change that does merge.

### Technical Risks

**Risk**: Module consensus bugs, complexity explosion **Mitigation**: Module isolation, formal verification, security audits

### Social Risks

**Risk**: Community rejection, fork wars, reputation attacks **Mitigation**: Focus on substance, build alternatives, let market decide; not asking permission, let code speak, coalition provides proof

### Ultimate Protection

Governance forks provide the ultimate accountability mechanism.

## 9. Conclusion

How the reference client changes is still a maintainer process. BLVM is an independent implementation of a written spec. Commons publishes the rules for changing the software, using Ostrom's principles for a commons and a signature schedule maintainers can fork.

This isn't speculation. It's applying battle-tested principles from economics, social science, and cryptography to governance. Each framework addresses weaknesses in the others: cryptography

makes Ostrom enforceable at scale, infrastructure enables Hayek's competition, and modularity plus fork-ability creates competitive discovery.

The foundation and architecture exist in public repositories. The project's future depends on community participation.

The repositories are public.

## References

### Academic Sources


- Ostrom, E. (1990). *Governing the Commons: The Evolution of Institutions for Collective Action*. Cambridge University Press.
- Hayek, F. A. (1945). The Use of Knowledge in Society. *American Economic Review*, 35(4), 519-530.
- De Filippi, P., & Loveluck, B. (2016). The Invisible Politics of Bitcoin: Governance Crisis of a Decentralized Infrastructure. *Internet Policy Review*, 5(3).
- Hough, E. (2025). "Funding the Peer-to-Peer Ethos: A Network Analysis Proposal of Bitcoin's Technical & Social Collectives." Unpublished research proposal, INFO 4360: Communication Networks and Social Capital, Cornell University.
- Apache Software Foundation. FY2024 Annual Report. https://www.apache.org/foundation/docs/FY2024AnnualReport.pdf
- Python Software Foundation. Form 990, fiscal year ending December 2024. ProPublica Nonprofit Explorer.
- Rust Foundation. Form 990, year ending December 31, 2024. https://rustfoundation.org/wp-content/uploads/2025/10/form-990-2024.pdf
- Zcash Foundation. Financial statements for the year ended December 31, 2024. https://zfnd.org/wp-content/uploads/2025/11/ZCash-24-FS-Final.pdf
- Polkadot Treasury. 2024 Polkadot Treasury Report. https://forum.polkadot.network/t/2024-polkadot-treasury-report/11658
- Linux Foundation. Form 990, fiscal year ending December 2024. Revenue about $221 million. ProPublica Nonprofit Explorer.
- Uzzi, B. (1997). Social Structure and Competition in Interfirm Networks: The Paradox of Embeddedness. *Administrative Science Quarterly*, 42(1), 35-67. https://doi.org/10.2307/2393808
- Arrow, K. J. (1951). *Social Choice and Individual Values*. Wiley.
- Gibbard, A. (1973). Manipulation of Voting Schemes: A General Result. *Econometrica*, 41(4), 587-601.
- Hirschman, A. O. (1970). *Exit, Voice, and Loyalty*. Harvard University Press.
- Satterthwaite, M. A. (1975). Strategy-Proofness and Arrow's Conditions. *Journal of Economic Theory*, 10(2), 187-217.


### Historical Sources


- Nakamoto, S. (2008). Bitcoin: A Peer-to-Peer Electronic Cash System. https://bitcoin.org/bitcoin.pdf
- Andresen, G. (2014). Bitcoin: The Future of Money? Princeton University, March 27, 2014.
- Hearn, M. (2016). The Resolution of the Bitcoin Experiment. *Medium*, January 15, 2016.


### Technical Sources

- Sharp, T. (2025). Hornet Node and the Hornet DSL: A Minimal, Executable Specification for Bitcoin Consensus. https://hornetnode.org/paper.html, https://arxiv.org/abs/2509.15754. Executable specification: https://hornetnode.org/spec
- McClelland, K. (2026). btc-verified: Verified Bitcoin protocol components in Lean 4. https://github.com/ProofOfKeags/btc-verified. See also: Formal Vibefication
- CVE-2018-17144 (2018). Bitcoin Core Denial of Service Vulnerability. *CVE Details*. https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2018-17144
- Bitcoin Core Statistics. Public GitHub repository data.
- Bitcoin Governance Research. Merge concentration, review concentration, and contributor timelines for Bitcoin Core. https://github.com/secsovereign/bitcoin-governance-research
- Bitcoin Optech Topics: High quality technical primers and references
  - Merged mining: https://en.bitcoin.it/wiki/Merged_mining_specification
  - Stratum v2: https://stratumprotocol.org/
- BOLT (Lightning) Specifications: https://github.com/lightning/bolts


**Repository Links**

- https://github.com/BTCDecoded/blvm-spec
- https://github.com/BTCDecoded/blvm-spec-lock (Orange Paper-to-Rust contract verification: `cargo spec-lock verify`)
- https://github.com/BTCDecoded/blvm-protocol
- https://github.com/BTCDecoded/blvm-consensus
- https://github.com/BTCDecoded/blvm-node
- https://github.com/BTCDecoded/blvm (packaging and release surface)
- https://github.com/BTCDecoded/blvm-sdk
- https://github.com/BTCDecoded/governance (governance configuration)
- https://github.com/BTCDecoded/blvm-commons (Governance App that enforces merge policy; a GitHub App is one way to run it)